

重大 AI 更新提醒

10-01 | llama.cpp 双版本 CUDA 优化落地，PyTorch 发布 Blackwell 上的 Jagged Flash Attention 实践

本期导读

本期聚焦推理基础设施的三条高可信更新：llama.cpp 连续发布两个 CUDA 相关版本，分别针对 Volta 架构的 K-quant 解码吞吐和 NVFP4 量化在 cuBLAS 路径上的计算类型处理；PyTorch 官方博客则公开了在 NVIDIA Blackwell（B200）上优化 Jagged Flash Attention 的工作，指向 SOTA FA4 方向。三条内容都直接关系到本地推理与注意力内核的性能表现，适合关注推理加速与 GPU 内核优化的开发者跟进。

已检查来源

GitHub Release、PyTorch Blog

1. llama.cpp b11335：将 sm70（Volta）路由到 Turing MMVQ nwarps 表，提升 K-quant 解码吞吐

来源 GitHub Release | 类型 GitHub Release | 主题 推理性能与基础设施 | 时间 10-01 22:08 | 分数 7

是什么

这是 llama.cpp 的一个 CUDA 后端修复版本。此前 Volta 架构（sm_70）没有自己的 MMVQ 参数表，会回退到 GENERIC 表，导致 K-quant 在 batch-1 解码（ncols_dst == 1）时以 nwarps=4 启动。该版本把 sm_70 在设备端和主机端的表选择器中都路由到已有的 MMVQ_PARAMETERS_TURING 表，因为 sm_70 与 Turing 共享调优参数，K-quant 的 vec_dot 在该表下更偏好 nwarps=2。

通俗解释

可以把 GPU 上的矩阵乘法想象成一支施工队：nwarps 就是同时派几个小组干活。Volta 这块老显卡以前没有专属的施工方案，只能套用通用方案，结果派了 4 个小组去搬一批本来只够 2 个小组高效搬运的货，反而互相等待。这个补丁给 Volta 换上了 Turing 的施工方案，改成派 2 个小组，搬得更顺。作者在 Tesla V100 上用 Qwen3.8-27B Q4_K_M 模型实测，生成速度从原来的水平提升了约 3.17%，也就是每秒多出约 1.091 个 token，而且困惑度完全不变，说明输出质量没有受影响。

主要作用

解决 Volta 架构 GPU 在运行 K-quant 量化模型做单批次解码时，因缺少专属 MMVQ 参数表而错误使用 nwarps=4 导致的性能损失。对于仍在 V100 等 Volta 显卡跑本地大模型的用户，这是一个无需改模型、无需改量化格式就能直接获得吞吐提升的修复。

核心功能

1) 为 sm_70 增加 MMVQ 参数表路由，设备端与主机端选择器同步修改；2) K-quant 的 mul_mat_vec_q 内核在补丁后以 nwarps=2 启动，而 Q4_0/Q8_0 仍保持 nwarps=4；3) 实测在 Tesla V100 32GB PCIe 上，Qwen3.8-27B Q4_K_M、tg128 场景下吞吐提升 3.17%（1.091 t/s），六个 ABBA 分块配对差值全部为正，t 值 49.0；4) 困惑度逐位一致（6.3697 0.04066，wiki.test.raw），说明精度无损。

对比判断

候选摘要中明确给出的对比是：在单张 Tesla V100 32GB PCIe（PG500-216，驱动 580.178.04，CUDA 12.0.140）上，使用 Qwen3.8-27B Q4_K_M、tg128、交错 A/B 的 6 组 ABBA 分块配对测试，吞吐提升 1.091 t/s，即 3.17%，t 值为 49.0，六个分块差值均为正；困惑度在两个构建之间逐位相同（6.3697 0.04066）。除此之外暂无可信公开对比数据。

适合谁

适合在 Volta 架构 GPU（如 Tesla V100）上部署 llama.cpp 做本地推理的开发者、运维人员和自建推理服务团队；也适合关注 CUDA 内核调优、K-quant 量化推理性能的研究者，以及需要在不牺牲困惑度的前提下榨取老卡吞吐的工程团队。

直达链接 <https://github.com/ggml-org/llama.cpp/releases/tag/b11335>

2. llama.cpp b11331 : 在 cuBLAS 路径上处理 NVFP4 的计算类型，量化模型默认改用 BF16 计算

来源 GitHub Release | 类型 GitHub Release | 主题 推理性能与基础设施 | 时间 10-01 19:48 | 分数 7

是什么	这是 llama.cpp 的另一个 CUDA 后端版本，主要处理 NVFP4 量化在 cuBLAS 路径上的计算类型问题。改动包括：在硬件允许时为量化模型使用 BF16 计算类型；针对 NVFP4 将累加精度设为 bf16，因为 NVFP4 至少需要 bf16 的数值范围；同时保留 per-expert matmul 的 op_params。该版本由 ynankani 提交，Johannes G 参与共同署名。
通俗解释	NVFP4 是一种把模型权重压到 4 位浮点的量化格式，好处是省显存、跑得快，但 4 位能表示的数字范围很窄，就像用一把刻度很粗的尺子量东西，稍微大一点的数就量不准。这个补丁的做法是：在真正做乘加运算时，把中间累加用的 尺子 换成刻度更细的 BF16，这样即使权重是 4 位的，累加过程也不会因为范围不够而溢出或丢精度。同时它只在硬件支持时才启用 BF16 计算，避免在老硬件上强行使用导致问题。
主要作用	解决 NVFP4 量化模型在 cuBLAS 路径上因计算类型不匹配而可能出现的数值范围不足、精度异常或运行失败问题。对于希望在支持 NVFP4 的硬件上运行 4 位量化大模型、又不想牺牲数值稳定性的用户，这个版本让 NVFP4 路径更可靠。
核心功能	1) 在 cuBLAS 路径上为 NVFP4 显式处理计算类型；2) 硬件允许时，量化模型统一使用 BF16 计算类型；3) NVFP4 的累加精度设为 bf16，以满足其至少需要 bf16 数值范围的要求；4) 保留 per-expert matmul 的 op_params，避免 MoE 类模型在专家矩阵乘法上丢失参数配置。
对比判断	候选摘要中未给出具体的 benchmark、速度或成本对比数据，因此暂无可信公开对比数据。
适合谁	适合使用 NVFP4 量化格式在 CUDA 上部署大模型的开发者与推理工程师，尤其是涉及 MoE (per-expert matmul) 结构的模型团队；也适合关注低比特量化数值稳定性、cuBLAS 计算类型选择的研究者和框架维护者。

直达链接 <https://github.com/ggml-org/llama.cpp/releases/tag/b11331>

3. PyTorch 博客：用 TLX 优化 Jagged Flash Attention，迈向 Blackwell 上的 SOTA FA4

来源 PyTorch Blog | 类型 官方发布 主题 开发者生态与数据工具 时间 10-01 17:26 | 分数 7

是什么	这是 PyTorch 官方博客的一篇文章，介绍在 NVIDIA Blackwell (B200) 上对 Jagged Flash Attention (JFA) 的优化工作。JFA 是 Meta 生成式广告模型 (GEM) 背后的注意力内核。文章围绕使用 TLX 来优化该内核展开，并将其定位为通往 Blackwell 上 SOTA FA4 的路线。
通俗解释	普通的注意力计算假设每个样本的序列长度都一样，像一排整齐的格子，方便批量处理。但广告推荐这类场景里，每个用户的序列长度参差不齐，有的很长有的很短，像锯齿一样高低不平，这就是 Jagged。如果硬按最长的那条来对齐，短序列就会塞满无用的填充，浪费算力。JFA 就是专门为这种长短不一的序列设计的注意力内核，而 TLX 是用于编写和优化这类 GPU 内核的工具。文章讲的是如何在最新的 Blackwell B200 上把这件事做得更快，目标是达到 FA4 级别的先进水平。
主要作用	解决变长序列 (jagged) 场景下注意力计算效率低的问题，尤其是 Meta GEM 这类生成式广告模型在 Blackwell 硬件上的性能瓶颈。对于需要处理长短不一序列的推荐、广告和生成式模型团队，这篇文章提供了在最新 GPU 上优化注意力内核的实践方向。
核心功能	1) 聚焦 Jagged Flash Attention (JFA) 这一面向变长序列的注意力内核；2) 使用 TLX 进行内核优化；3) 目标平台为 NVIDIA Blackwell (B200)；4) 以通往 SOTA FA4 为优化路线，服务于 Meta 生成式广告模型 (GEM)。
对比判断	候选摘要提到该工作指向 Blackwell 上的 SOTA FA4，并含可比较性能 成本线索，但摘要中未给出具体 benchmark 数值、速度提升幅度或成本对比，因此暂无可信公开对比数据。
适合谁	适合从事注意力内核优化、GPU 性能工程的开发者与研究者；也适合在推荐、广告、生成式模型中使用变长序列注意力、并计划迁移到 NVIDIA Blackwell (B200) 的工程团队，以及关注 PyTorch 生态内核优化实践的技术人员。

直达链接 <https://pytorch.org/blog/optimizing-jagged-flash-attention-with-tlx-the-road-toward-sota-fa4-on-blackwell/>

本期内容均来自 GitHub Release 与 PyTorch 官方博客，未引入候选之外的任何数据；具体性能数字以对应链接中的原始说明为准。